MLOps Assignment 1 - Heart Disease Risk Prediction

Name: Amarnath K

BITS ID: 2025ae05664
Course: Machine Learning Operations (AIMLCZG523)
Code repository: https://github.com/Amarnath2016/heart-mlops

Video Link:
https://drive.google.com/file/d/1LZKLkNhmfJih0lWMIoTivp8zB6t41_KP/view?usp=drive_link

1. Setup and installation

Python 3.11 is used. From a clean checkout:

- python -m venv .venv && source .venv/bin/activate
- pip install -r requirements.txt
- python scripts/download_data.py # downloads UCI data, writes data/processed/heart_clean.csv
- python -m src.eda # figures in reports/figures
- python -m src.train # MLflow runs in ./mlruns, model in models/
- mlflow ui --backend-store-uri ./mlruns # browse experiments
- pytest -v
- docker build -t heart-api . && docker run -p 8000:8000 heart-api
- kubectl apply -f k8s/

2. Data and exploratory analysis

The Cleveland subset of the UCI Heart Disease dataset has 303 rows and 13 predictors plus a 0-4 diagnosis column. The diagnosis was binarised (0 = no disease, 1-4 = disease). After removing duplicates, 303 rows remain: 164 without and 139 with disease (45.9% positive), so the classes are reasonably balanced and accuracy is meaningful alongside recall and ROC-AUC.

Missing values ('?' in the raw file): ca (4), thal (2). They are imputed inside the modelling pipeline (median for numeric, most-frequent for categorical) so that statistics are learned on training folds only.

Features most correlated with the target (Pearson, |r|): thal (+0.53), ca (+0.46), exang (+0.43), oldpeak (+0.42), thalach (-0.42).

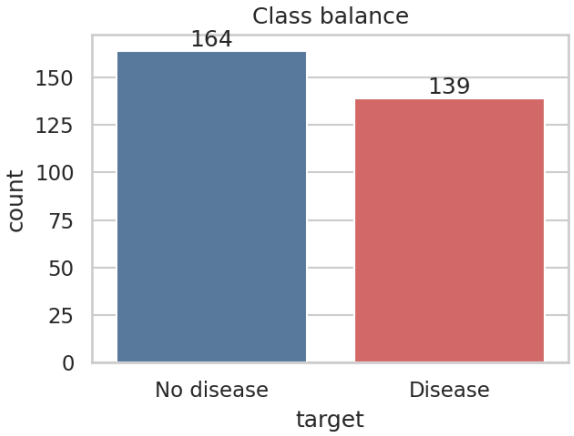


Figure 1 - Class balance

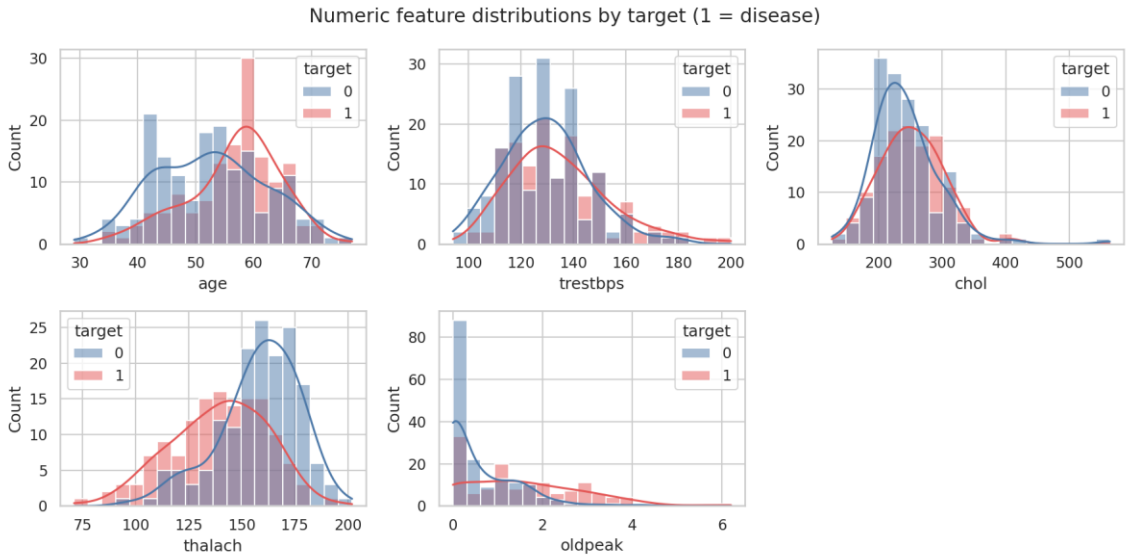


Figure 2 - Numeric feature distributions by target

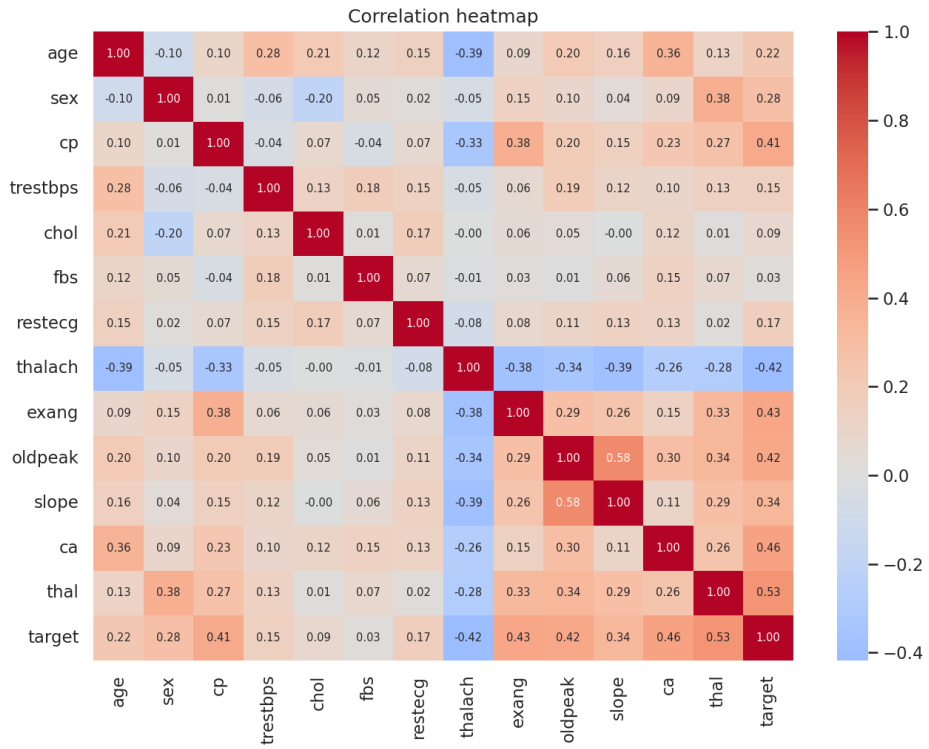


Figure 3 - Correlation heatmap

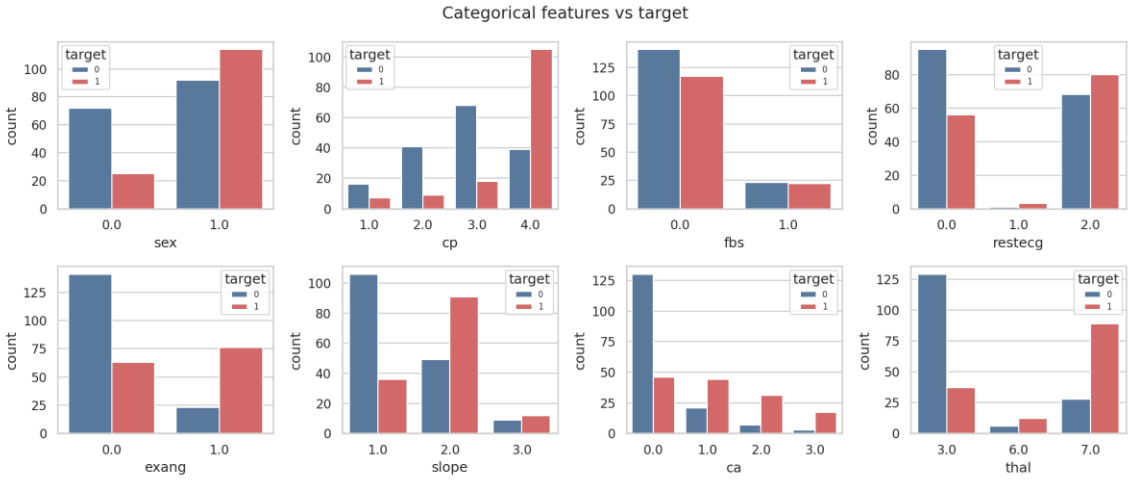


Figure 4 - Categorical features vs target

3. Feature engineering and model development

A single scikit-learn ColumnTransformer is used for both training and serving: numeric features (age, trestbps, chol, thalach, oldpeak) are median-imputed and standardised; categorical codes (sex, cp, fbs, restecg, exang, slope, ca, thal) are mode-imputed and one-hot encoded with

handle_unknown='ignore'. Because the transformer is part of the model pipeline, the API only needs the raw 13 fields.

Data was split 80/20 (stratified, seed 42). Logistic Regression and Random Forest were tuned with GridSearchCV (5-fold stratified CV, ROC-AUC as the objective) on the training split only. The final model is chosen by mean cross-validated ROC-AUC; the hold-out test set is used only for the final report.

Model	CV ROC-AUC	Test acc.	Test precision	Test recall	Test ROC-AUC
logistic_regression	0.902	0.885	0.839	0.929	0.966
random_forest	0.902	0.852	0.806	0.893	0.948

Selected model: logistic_regression with parameters {'clf__C': '1.0'}.

Cross-validated metrics of the selected model (mean +/- std): accuracy 0.847 +/- 0.010; precision 0.877 +/- 0.054; recall 0.783 +/- 0.046; f1 0.825 +/- 0.007; roc_auc 0.902 +/- 0.014.

Note: with ~240 training rows, test-set metrics have wide uncertainty; differences of a few points between models are not conclusive.

4. Experiment tracking

Each model family is one MLflow run in experiment 'heart-disease-risk'. Logged: best hyper-parameters, grid size, split sizes, CV mean/std and hold-out metrics, ROC curve and confusion-matrix plots, and the fitted pipeline as an MLflow model. Selected run id: 8d3f024d90894143aff79a1f838e448b.

5. Packaging and reproducibility

The full pipeline (preprocessing + classifier) is saved with joblib to models/model.joblib with models/model_metadata.json (features, params, metrics, run id). Dependencies are pinned in requirements.txt; the serving image uses requirements-serve.txt with identical versions. Random seeds are fixed.

6. CI/CD and automated testing

Unit tests (tests/) cover data parsing and cleaning, preprocessing output, stratified splitting, unseen categories, metric ranges, input validation and the API (health, predict, bad input, metrics). The GitHub Actions workflow runs flake8, pytest, data download + EDA + training, uploads model/MLflow/reports as artifacts, then builds the Docker image and smoke-tests /health and /predict. Any failing step fails the run with its logs.

7. Containerisation

The Flask API runs under gunicorn in a python:3.11-slim image as a non-root user. POST /predict accepts the 13 fields as JSON and returns prediction, label, confidence and probability of heart disease. Other endpoints: /health, /model-info, /metrics.

```
{"age":63,"sex":1,"cp":4,"trestbps":145,"chol":233,"fbs":1,"restecg":2,"thalach":150,"exang":0,"oldpeak":2.3,"slope":3,"ca":0,"thal":6}
```

8. Production deployment

k8s/deployment.yaml (2 replicas, readiness/liveness probes, resource limits) and k8s/service.yaml (type LoadBalancer) are applied to a local Kubernetes cluster; an optional Ingress is provided. Access instructions are in the README.

9. Monitoring and logging

Every request is logged as one JSON line (timestamp, request id, method, path, status, latency, prediction). /metrics exposes request counts by endpoint/status, predictions per class and a latency histogram; Prometheus scrapes it and a provisioned Grafana dashboard shows request rate, p95 latency, prediction mix and error rate (monitoring/docker-compose.yml).

10. Architecture

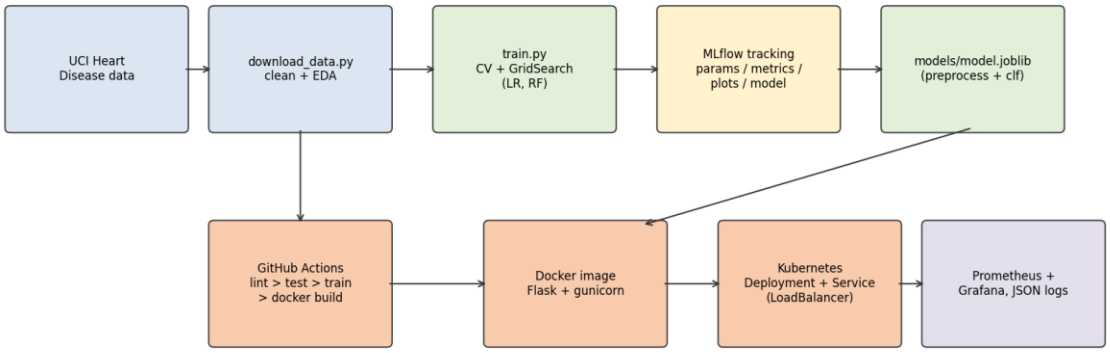
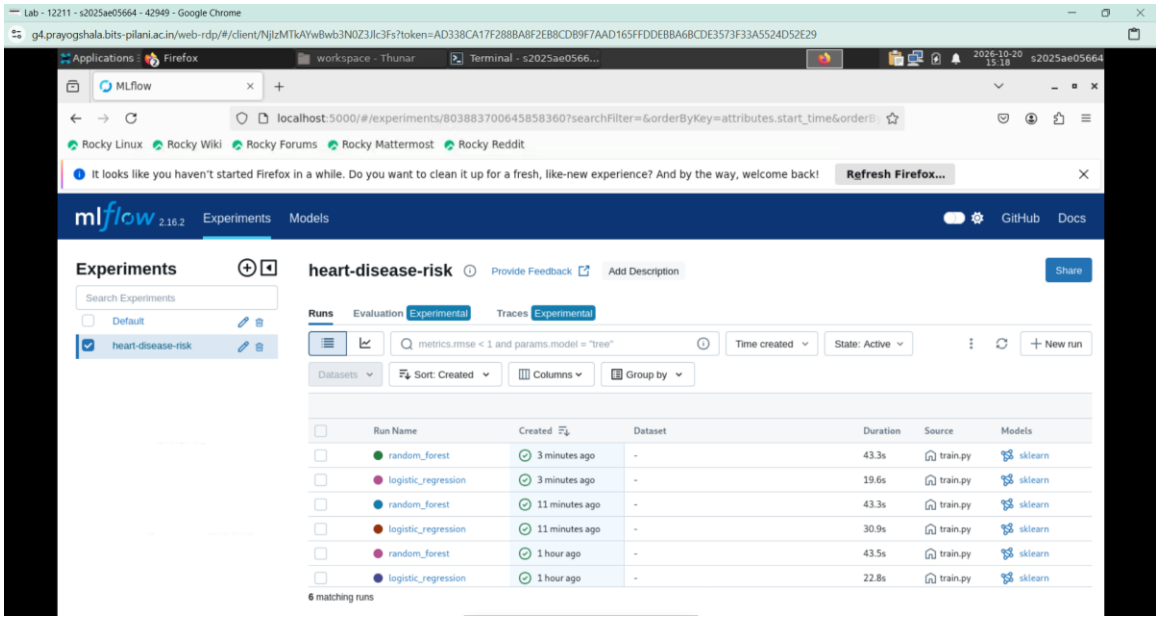
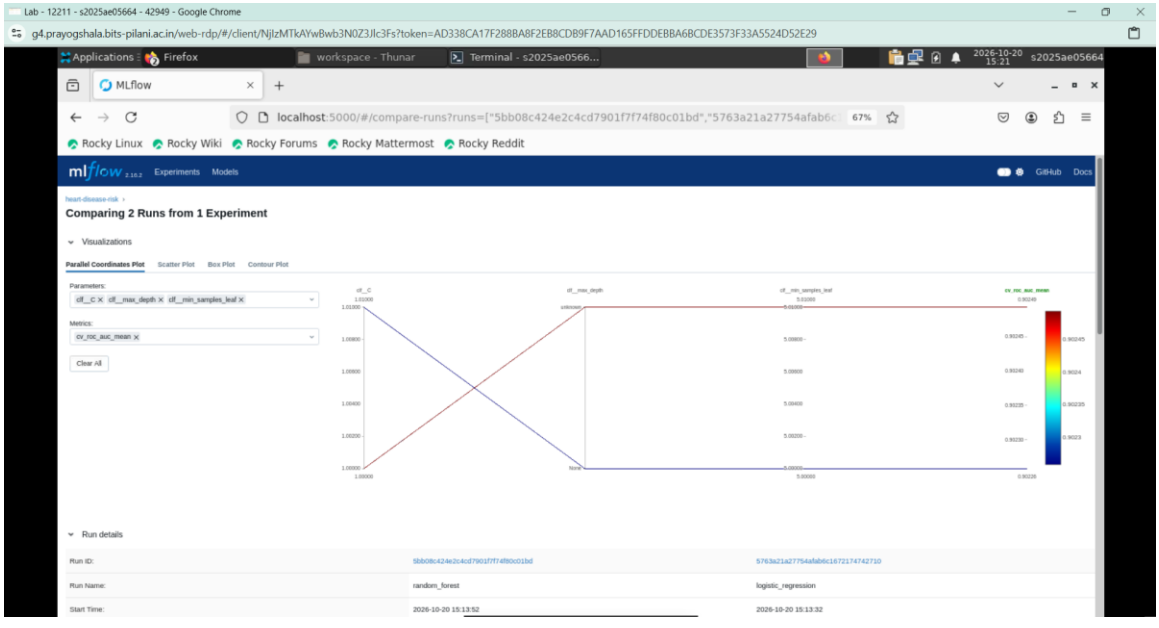


Figure - End-to-end architecture

11. Screenshots



MLflow UI - experiment runs



MLflow - run comparison

```
=============== 4. Commit and push to GitHub ===============
[main 96d97b0] Add pipeline results, cleaned data, trained model
 Committer: s2025ae05664 <s2025ae05664@ip-10-3-22-41.ap-south-1.compute.internal>
Your name and email address were configured automatically based
on your username and hostname. Please check that they are accurate.
You can suppress this message by setting them explicitly. Run the
following command and follow the instructions in your editor to edit
your configuration file:

    git config --global --edit

After doing this, you may fix the identity used for this commit with:

    git commit --amend --reset-author

 3 files changed, 2 insertions(+), 2 deletions(-)
 create mode 100644 reports/MLOps_Assignment_Report.docx
 create mode 100644 reports/figures/architecture.png
  Push to GitHub now? (needs username + personal access token) [y/N] y
```

GitHub Actions - successful workflow run

```
[main ab0c495] Add pipeline results, cleaned data, trained model
 Committer: s2025ae05664 <s2025ae05664@ip-10-3-17-124.ap-south-1.compute.internal>
Your name and email address were configured automatically based
on your username and hostname. Please check that they are accurate.
You can suppress this message by setting them explicitly. Run the
following command and follow the instructions in your editor to edit
your configuration file:

    git config --global --edit

After doing this, you may fix the identity used for this commit with:

    git commit --amend --reset-author

 1 file changed, 2 insertions(+), 2 deletions(-)
  Push to GitHub now? (needs username + personal access token) [y/N] y
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 4 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 446 bytes | 148.00 KiB/s, done.
Total 4 (delta 3), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To https://github.com/Amarnath2016/heart-mlops.git
   ac3ac59..ab0c495  main -> main
  -> Now open https://github.com/Amarnath2016/heart-mlops/actions and wait for the run to finish
```

Git uploaded artifacts

```
=============== 5. Docker image and container ===============
[+] Building 43.6s (12/12) FINISHED                                                
```

```
ef4d1f083f93555c24341d1b27235d225c899c5b70bdd725101bf25938d8f40e

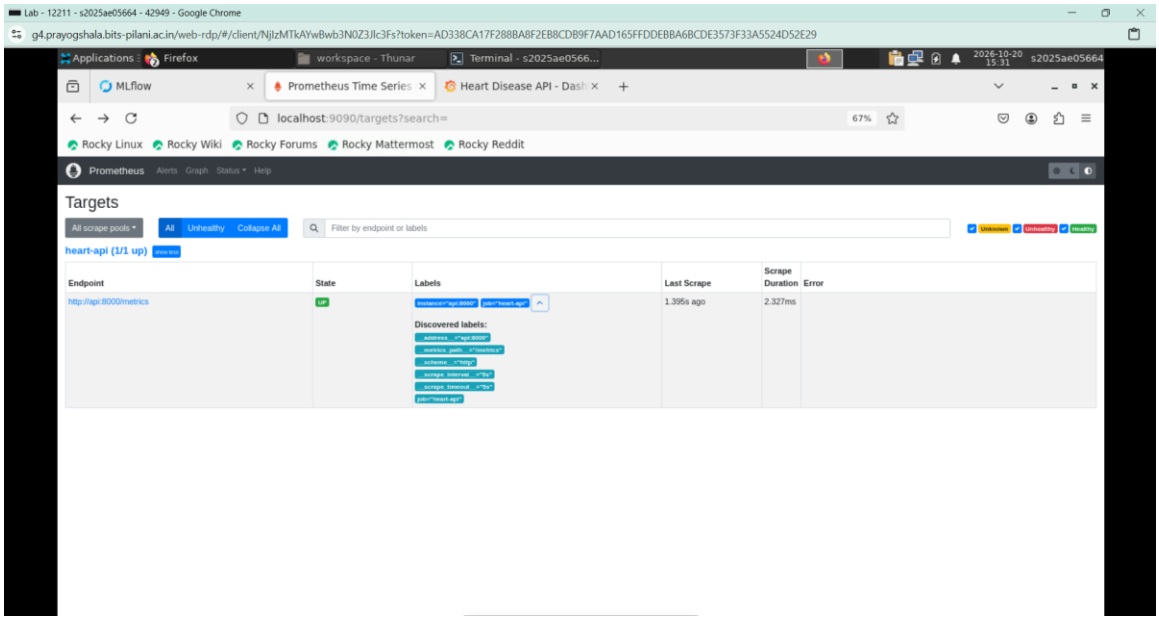
--- /predict ---
./run_pipeline.sh: line 111: scripts/sample_request.sh: Permission denied

--- docker logs ---
[2026-10-20 09:55:38 +0000] [1] [INFO] Starting gunicorn 22.0.0
[2026-10-20 09:55:38 +0000] [1] [INFO] Listening at: http://0.0.0.0:8000 (1)
[2026-10-20 09:55:38 +0000] [1] [INFO] Using worker: gthread
[2026-10-20 09:55:38 +0000] [8] [INFO] Booting worker with pid: 8
{"ts": "2026-10-20T09:55:40+0000", "request_id": "570165e450e7", "method": "GET", "path": "/health", "status": 200, "latency_ms": 0.22,
 "remote": "172.17.0.1"}
172.17.0.1 - - [20/Oct/2026:09:55:40 +0000] "GET /health HTTP/1.1" 200 16 "-" "curl/7.76.1"
```

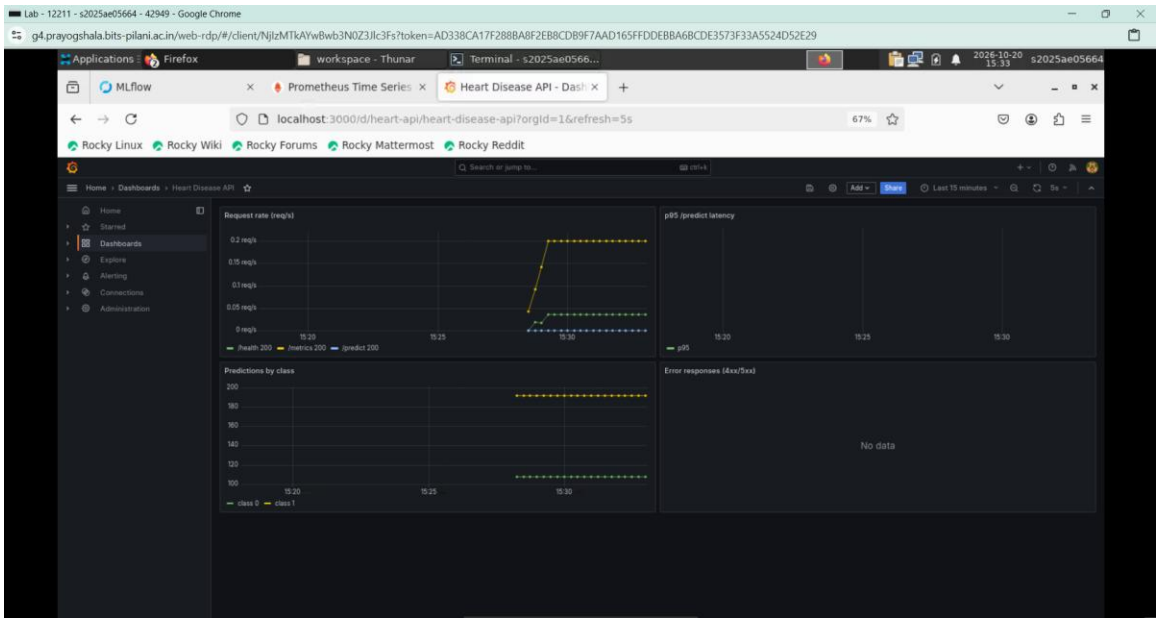
Container running and answering /predict, deployment status

```
{"ts": "2026-10-20T09:55:40+0000", "request_id": "570165e450e7", "method": "GET", "path": "/health", "status": 200, "latency_ms": 0.22,
 "remote": "172.17.0.1"}
172.17.0.1 - - [20/Oct/2026:09:55:40 +0000] "GET /health HTTP/1.1" 200 16 "-" "curl/7.76.1"
```

Structured request logs



Prometheus target UP



Grafana dashboard

12. Repository

https://github.com/Amarnath2016/heart-mlops

13. Video Link

https://drive.google.com/file/d/1LZKLkNhmfJih0lWMIoTivp8zB6t41_KP/view?usp=drive_link